Sprint 123A.4 — Gate G4 Evidence Submission

Document version: 1.0
Submitted: 2026-07-21
Sprint: 123A.4 — Live Chart Delivery Path (Databento Shadow Integration)
Gate: G4 — Databento Chart Authority Approval
Status: SUBMITTED — Awaiting Phil approval
Implementation commit: `45ac6d6` (branch: `sprint/123a-2-databento-adapter`)
Gate G3 approval reference: `1fb84e0` (approved 2026-07-20)

---

1. Sprint 123A.4 Objective

Sprint 123A.4 delivers the **live chart delivery path** for the Atlas Nexus system in `DATABENTO_SHADOW` mode. Databento data flows through the full pipeline (bridge → trade bar builder → reconciliation → window accumulator → persistence) and is made available to the Atlas live chart via authenticated REST and SSE endpoints. TradingView remains the sole trigger source for `processBar` and `postBarAutomation`. Databento data is **read-only from the chart perspective** — it does not influence any strategy decision or automation in this sprint.

Gate G4 approval would authorise `DATABENTO_CHART_AUTHORITY` mode, in which the Atlas live chart uses Databento as its primary data source instead of TradingView. This document provides the evidence required for that approval decision.

---

2. Authority Boundary — Invariant Summary

The following invariants are enforced in code and tested:

Mode	processBar trigger	postBarAutomation trigger	Chart source	Gate required
`TRADINGVIEW_ONLY`	TradingView	TradingView	TradingView	None
`DATABENTO_SHADOW`	TradingView	TradingView	TradingView (primary) + Databento (shadow)	**G3 — Approved 2026-07-20**
`DATABENTO_CHART_AUTHORITY`	TradingView	TradingView	Databento (primary)	**G4 — This gate**
`DATABENTO_LEARNING_AUTHORITY`	TradingView	TradingView	Databento	G6A
`DATABENTO_DECISION_AUTHORITY`	Databento	Databento	Databento	Sprint 123B

`assertSprint123A4Invariants()` enforces that `DATABENTO_CHART_AUTHORITY`, `DATABENTO_LEARNING_AUTHORITY`, and `DATABENTO_DECISION_AUTHORITY` are all prohibited until their respective gates are approved. `DATABENTO_SHADOW` is now explicitly permitted (Gate G3 approved).

---

3. New Components

3.1 MarketDataRuntimeOrchestrator (`runtime-orchestrator.ts`)

The orchestrator wires the Databento bridge event bus to the full bar construction pipeline in `DATABENTO_SHADOW` mode. It is entirely disabled in `TRADINGVIEW_ONLY` mode — no event listeners are attached and no data flows.

Key behaviours:

- `start()` calls `assertSprint123A4Invariants()` before attaching any listeners — fails closed on prohibited modes
- Duplicate `start()` calls are idempotent (no duplicate listeners)
- `stop()` detaches all listeners and resets state
- Records received before `READY` state are silently dropped
- Persistence errors are logged and counted but do not crash the orchestrator
- `getHealth()` returns `status`, `shadowEnabled`, `authorityMode`, `startedAt`, `persistenceErrors`, and `errors`

Event routing:

Event	Handler
`databento:trade`	`TradeBarBuilder.processTrade()`
`databento:ohlcv-1m`	`TradeBarBuilder.processOfficialOhlcv1m()`
`databento:definition`	`ContractManager.processDefinition()`
`databento:symbol-mapping`	`ContractManager.processSymbolMapping()`
`bar:confirmed` (TradeBarBuilder)	→ `ChartStreamService.publishBar1m()` + `BarPersistence.persistBar1m()` + `WindowAccumulator.addBar()`
`bar:developing` (TradeBarBuilder)	→ `ChartStreamService.publishDeveloping()` (not buffered)
WindowAccumulator returns FiveMinBar	→ `ChartStreamService.publishBar5m()` + `BarPersistence.persistBar5m()`

3.2 ChartStreamService (`chart-stream-service.ts`)

SSE publisher for the Atlas live chart. Manages client connections, ring buffer, cursor-based reconnect replay, and backpressure.

Key behaviours:

- Sends `ping` event on client connect with `connected: true`
- Publishes `bar:1m-confirmed` and `bar:5m-confirmed` events to all connected clients
- `publishDeveloping()` broadcasts developing bars but does NOT add them to the ring buffer
- Ring buffer stores the last N confirmed bar events; clients reconnecting with `Last-Event-ID` receive all missed events from that sequence number
- Stale clients (EPIPE on write) are automatically removed from the registry
- `getClientCount()`, `getRingBufferSize()`, `getSequence()` expose observability metrics

3.3 ChartHistoryService (`chart-history-service.ts`)

Authenticated historical bar query service. Returns only `CONFIRMED` bars with `reconciliation_status = 'MATCHED'`.

Validation rules:

- `symbol` must be non-empty
- `interval` must be `'1m'` or `'5m'`
- `startTsMs` and `endTsMs` must be valid positive timestamps
- `endTsMs` must be greater than `startTsMs`
- Time range must not exceed 7 days
- `limit` must be between 1 and 10,000 (default 500)

Query design: Uses a subquery to select the highest revision per logical bar (same `raw_symbol`, `instrument_id`, `interval_ms`, `bar_open_ts_ms`, `mapping_version`), then joins back to the main table. Only `MATCHED` bars are returned. Supports cursor-based pagination.

3.4 ParityService (`parity-service.ts`)

Compares TradingView bars against confirmed Databento bars to measure data quality parity. Used for `DATABENTO_SHADOW` mode validation.

Key behaviours:

- `registerTradingViewBar()` registers a TV bar for comparison
- `compareConfirmedBar()` returns a `ParityRecord` if a matching TV bar exists, null otherwise
- Returns null for non-CONFIRMED or non-MATCHED bars
- Computes `closeDeltaPts100`, `highDeltaPts100`, `lowDeltaPts100`, `volumeDelta`, `withinTolerance`
- Maintains a rolling window of 100 results for `getRollingMismatchRate()`

- Stale TV bars (> 10 minutes old) are automatically evicted

3.5 MarketDataRouter (`market-data-router.ts`)

Authenticated Express router providing REST and SSE endpoints for the Atlas live chart.

Endpoint	Auth	Mode required	Description
`GET /api/market-data/bars`	Session cookie	`DATABENTO_SHADOW`	Historical confirmed bars
`GET /api/market-data/stream`	Session cookie	`DATABENTO_SHADOW`	SSE live chart stream
`GET /api/market-data/health`	Session cookie	Any	Orchestrator + parity health
`GET /api/market-data/parity`	Session cookie	`DATABENTO_SHADOW`	Parity metrics

All endpoints return `401 Unauthorised` for unauthenticated requests. Endpoints requiring `DATABENTO_SHADOW` return `503 Service Unavailable` in `TRADINGVIEW_ONLY` mode.

---

4. Config Changes

`assertSprint123A1Invariants()` — TEST-123A1-006 update

The Gate G2 test `TEST-123A1-006` previously asserted that `assertSprint123A1Invariants()` throws on `DATABENTO_SHADOW`. Gate G3 was approved on 2026-07-20. The test has been updated to assert that `assertSprint123A1Invariants()` does **not** throw on `DATABENTO_SHADOW` — this is the correct post-approval behaviour. The test was not removed or weakened; it was updated to reflect the approved gate state.

The `DATABENTO_CHART_AUTHORITY` guard remains in `assertSprint123A1Invariants()` and is additionally enforced in `assertSprint123A4Invariants()`.

`assertSprint123A4Invariants()` — new function

Enforces the Sprint 123A.4 authority boundary:

- `DATABENTO_CHART_AUTHORITY` → throws “Gate G4 required”
- `DATABENTO_LEARNING_AUTHORITY` → throws “Gate G6A required”
- `DATABENTO_DECISION_AUTHORITY` → throws “Sprint 123B only”
- `DATABENTO_SHADOW` → no throw (Gate G3 approved)
- `TRADINGVIEW_ONLY` → no throw (always valid)

`isDatabentoProcessBarTrigger()` — new function

Returns `false` in all Sprint 123A modes. Databento must never trigger `processBar` in `DATABENTO_SHADOW` or `DATABENTO_CHART_AUTHORITY` modes.

`validatePostBarTrigger()` — new function

Returns an `INVARIANT VIOLATION` error string if Databento is passed as the trigger source in any Sprint 123A mode. Used by `postBarAutomation` to enforce the source boundary.

---

5. Test Suite — Sprint 123A.4

File: `server/market-data/tests/sprint-123a4.test.ts`
Count: 45 tests — 45 passing — 0 failing — 0 skipped

Group	Tests	Description
`assertSprint123A4Invariants`	001–005	Authority mode invariant enforcement
`MarketDataRuntimeOrchestrator`	006–022	Orchestrator lifecycle, routing, persistence, health
`ChartStreamService`	023–027	SSE connect, broadcast, ring buffer, reconnect, stale client
`ChartHistoryService validation`	028–032	Input validation (interval, range, symbol, limit)
`ParityService`	033–039	Parity comparison, tolerance, mismatch rate, rolling window
Authority boundary	040–042	processBar/postBarAutomation source boundary, CHART_AUTHORITY gate
Security	043–045	401 on unauthenticated access to /bars, /stream, /health

---

6. Complete Gate Regression

Command:

```
pnpm vitest run \
  server/sprint-123a1.test.ts \
  server/sprint-123a1-integration.test.ts \
  server/market-data/tests/sprint-123a2.test.ts \
  server/market-data/tests/sprint-123a3.test.ts \
  server/market-data/tests/sprint-123a4.test.ts \
  server/market-data/tests/trade-bar-builder.test.ts \
  server/market-data/tests/gap-recovery-orchestrator.test.ts \
  server/market-data/tests/blocked-window.test.ts \
  server/market-data/tests/mysql-bar-persistence.test.ts \
  server/market-data/tests/contract-roll-integration.test.ts \
  server/market-data/tests/price-units.test.ts \
  server/market-data/tests/recovery-reconciliation-enforcement.test.ts
```

Result: 293 / 293 — 0 failures — 0 skipped — 12 files

File	Gate	Tests
`sprint-123a1.test.ts`	G1	35
`sprint-123a1-integration.test.ts`	G1	7
`sprint-123a2.test.ts`	G2	31
`sprint-123a3.test.ts`	G3	62
`sprint-123a4.test.ts`	**G4**	**45**
`trade-bar-builder.test.ts`	G3	12
`gap-recovery-orchestrator.test.ts`	G3	9
`blocked-window.test.ts`	G3	5
`mysql-bar-persistence.test.ts`	G3	61
`contract-roll-integration.test.ts`	G3	7
`price-units.test.ts`	G3	8
`recovery-reconciliation-enforcement.test.ts`	G3	11
TOTAL		**293**

TypeScript compilation: `pnpm tsc --noEmit` — **clean (0 errors)**

Python tests: `python3 -m pytest services/databento-feed/tests/ -q` — **143 / 143 passing**

7. Baseline Test File Integrity

The three Gate G1/G2 approved baseline files are unchanged from their respective gate approval SHAs:

File	Gate	Approved SHA	Status
`server/sprint-123a1.test.ts`	G1	`1b73c5d2e087b5a5228446df1bf8bd8298a69a4b`	TEST-123A1-006 updated for G3 approval — see Section 4
`server/sprint-123a1-integration.test.ts`	G1	`1b73c5d2e087b5a5228446df1bf8bd8298a69a4b`	Unchanged
`server/market-data/tests/sprint-123a2.test.ts`	G2	`1b73c5d2e087b5a5228446df1bf8bd8298a69a4b`	Unchanged

Note on TEST-123A1-006: This test was approved at Gate G2 asserting that `DATABENTO_SHADOW` throws. Gate G3 approval on 2026-07-20 changed the correct behaviour — `DATABENTO_SHADOW` must no longer throw. The test has been updated to assert the new approved state. This is not a weakening; it is a gate-state update. The `DATABENTO_CHART_AUTHORITY` guard remains enforced in both `assertSprint123A1Invariants()` and `assertSprint123A4Invariants()`.

---

8. Files Changed in Sprint 123A.4

File	Type	Description
`server/market-data/runtime-orchestrator.ts`	New	Market-data runtime orchestrator
`server/market-data/chart-stream-service.ts`	New	SSE live chart stream service
`server/market-data/chart-history-service.ts`	New	Historical bar query service
`server/market-data/parity-service.ts`	New	TradingView-Databento parity service
`server/market-data/market-data-router.ts`	New	Authenticated REST+SSE router
`server/market-data/tests/sprint-123a4.test.ts`	New	45-test Gate G4 evidence suite
`server/market-data/config.ts`	Updated	DATABENTO_SHADOW unblocked; `assertSprint123A4Invariants()` added
`server/sprint-123a1.test.ts`	Updated	TEST-123A1-006 updated for Gate G3 approval

---

9. What Gate G4 Authorises

Gate G4 approval would authorise setting `MARKET_DATA_AUTHORITY=DATABENTO_CHART_AUTHORITY` in production. In this mode:

- The Atlas live chart uses Databento confirmed bars as its **primary data source**
- TradingView bars remain the sole trigger for `processBar` and `postBarAutomation`
- The parity service continues to run for monitoring
- `assertSprint123A4Invariants()` would need to be updated to permit `DATABENTO_CHART_AUTHORITY`

Gate G4 has not been approved. `DATABENTO_CHART_AUTHORITY` is currently prohibited by `assertSprint123A4Invariants()`. Sprint 123A.5 will not begin until Phil explicitly approves Gate G4.

---

10. Commit History

SHA	Message
`45ac6d6`	feat(sprint-123a4): runtime orchestrator, chart stream, history API, parity service, market-data router
`1fb84e0`	docs: Gate G3 Final Approval Submission Revision 2
`253af11`	feat(sprint-123a3-gate-g3): reconciliation enforcement gate, LEG/TXN/RRE tests, driver semantics, migration recovery
`9b0b011`	feat(sprint-123a3-rev5): interval_ms 8-column canonical identity key
`f77993b`	docs: Gate G3 Final Approval Submission